GOOGLE AI STUDIO MASTER BUILD DOCUMENT
PROJECT: FASHION AURA ORDER MANAGEMENT SYSTEM (ফ্যাশন আউরা অর্ডার ম্যানেজমেন্ট)

SYSTEM INSTRUCTION FOR GOOGLE AI STUDIO CODE GENERATOR
You are building a complete, production-ready, offline-first React + TypeScript + Tailwind CSS + IndexedDB (Dexie) application for "Fashion Aura" (ফ্যাশন আউরা), a women's clothing e-commerce business in Bangladesh.

When generating the application from this PDF:
1. Synchronize all 5 Stages below into a cohesive, runnable React + TypeScript codebase.
2. Preserve all 5 independent order dimensions (`orderType`, `priority`, `paymentStatus`, `fulfillmentStatus`, and derived `deliverySchedule`).
3. Ensure full Bangla (`bn` default) and English (`en`) support with Bangla digit conversion (`০-৯` <-> `0-9`) and BDT (`৳`) integer math.
4. Keep 100% offline capability via IndexedDB (`Dexie`) with zero fake/mock data in production views.
5. Wire the Dashboard, Universal Search, Combinable Filters, Order Form, Smart Paste Parser, Courier Template Engine, Backup/Export (CSV/JSON), Print Slip, and Trash/Audit logs together in `src/App.tsx`.

STAGE 1: DEPENDENCIES, DATA MODEL & INDEXEDDB (DEXIE)

File: `package.json`
```
{
  "name": "fashion-aura-oms",
  "private": true,
  "version": "1.0.0",
  "type": "module",
  "scripts": {
    "dev": "vite",
    "build": "tsc && vite build",
    "preview": "vite preview"
  },
  "dependencies": {
    "dexie": "^4.0.8",
    "dexie-react-hooks": "^1.1.7",
    "lucide-react": "^0.446.0",
    "react": "^18.3.1",
    "react-dom": "^18.3.1"
  },
  "devDependencies": {
    "@types/react": "^18.3.10",
```

```
    "@types/react-dom": "^18.3.0",
    "@vitejs/plugin-react": "^4.3.2",
    "autoprefixer": "^10.4.20",
    "postcss": "^8.4.47",
    "tailwindcss": "^3.4.13",
    "typescript": "^5.6.2",
    "vite": "^5.4.8"
  }
}
```

File: `src/types/models.ts`

```
export type OrderType = 'Regular' | 'Pre-order' | 'Free' | 'Exchange';
export type OrderPriority = 'Normal' | 'Urgent';
export type PaymentStatus =
  | 'Unpaid'
  | 'Partially Paid'
  | 'Paid'
  | 'Refund Pending'
  | 'Partially Refunded'
  | 'Refunded';
export type FulfillmentStatus =
  | 'New'
  | 'Confirmed'
  | 'Preparing'
  | 'Ready to Ship'
  | 'Shipped'
  | 'In Transit'
  | 'Delivered'
  | 'Delivery Failed'
  | 'Return Requested'
  | 'Returned'
  | 'Exchange Processing'
  | 'Cancelled';

export type DeliveryScheduleState = 'Unscheduled' | 'Scheduled';
export type PaymentEntryType = 'advance' | 'payment' | 'refund' | 'adjustment';
export type PaymentMethod = 'Cash/COD' | 'bKash' | 'Nagad' | 'Rocket' | 'Bank' | 'Other';
export type OrderSource = 'Facebook' | 'Meta Business Suite' | 'WhatsApp' | 'Repeat Customer' |
'Other';
export type CourierName = 'Steadfast' | 'Pathao' | 'RedX' | 'Paperfly' | 'Sundarban' | 'Other';
export type ProductSize = '36' | '38' | '40' | '42' | '44' | '46' | '48' | 'Other';
export type SyncState = 'pending' | 'syncing' | 'synced' | 'error';

export interface OrderItemRecord {
```

```
  id: string;
  orderId: string;
  productName: string;
  sku?: string;
  size: ProductSize;
  customSizeText?: string;
  quantity: number;
  unitPrice: number;
  color?: string;
  productNotes?: string;
  imageDataUrls?: string[];
}

export interface OrderRecord {
  id: string;
  orderNumber: string;
  customerId: string;
  customerName: string;
  primaryPhone: string;
  alternatePhone?: string;
  fullAddress: string;
  areaVillageStreet?: string;
  district: string;
  thanaUpazila: string;
  landmark?: string;
  orderType: OrderType;
  priority: OrderPriority;
  paymentStatus: PaymentStatus;
  fulfillmentStatus: FulfillmentStatus;
  orderDate: string;
  requestedDeliveryDate?: string;
  requestedDeliveryTimeWindow?: string;
  earliestDispatchDate?: string;
  preorderAvailabilityDate?: string;
  actualShipDate?: string;
  actualDeliveryDate?: string;
  assignedEmployeeName?: string;
  source: OrderSource;
  courier: CourierName;
  trackingNumber?: string;
  subtotal: number;
  deliveryCharge: number;
  additionalCharge: number;
  discount: number;
```

```
  totalBill: number;
  netPaid: number;
  outstanding: number;
  customerDeliveryInstructions?: string;
  internalStaffNotes?: string;
  exchangeReturnNotes?: string;
  freeOrderReason?: string;
  dispatchOverrideReason?: string;
  isDraft: boolean;
  isDeleted: boolean;
  version: number;
  createdAt: string;
  createdBy: string;
  updatedAt: string;
  updatedBy: string;
  syncState: SyncState;
}

export interface CustomerRecord {
  id: string;
  name: string;
  primaryPhone: string;
  defaultAddress: string;
  district: string;
  thanaUpazila: string;
  orderCount: number;
  totalSpent: number;
  returnCount: number;
  cancelCount: number;
  isRisky: boolean;
  riskReason?: string;
  updatedAt: string;
}

export interface PaymentLedgerRecord {
  id: string;
  orderId: string;
  type: PaymentEntryType;
  amount: number;
  method: PaymentMethod;
  trxId?: string;
  recordedBy: string;
  recordedAt: string;
}
```

```
export interface AuditLogRecord {
  id: string;
  entityType: 'order' | 'payment' | 'backup' | 'share';
  entityId: string;
  action: string;
  details: string;
  performedBy: string;
  performedAt: string;
}

export function getDeliveryScheduleState(
  order: Pick<OrderRecord, 'requestedDeliveryDate' | 'earliestDispatchDate'>
): DeliveryScheduleState {
  return Boolean(order.requestedDeliveryDate?.trim() || order.earliestDispatchDate?.trim())
    ? 'Scheduled'
    : 'Unscheduled';
}

### File: `src/db/database.ts`
import Dexie, { type Table } from 'dexie';
import type {
  OrderRecord,
  OrderItemRecord,
  CustomerRecord,
  PaymentLedgerRecord,
  AuditLogRecord
} from '../types/models';

export class FashionAuraDB extends Dexie {
  orders!: Table<OrderRecord, string>;
  orderItems!: Table<OrderItemRecord, string>;
  customers!: Table<CustomerRecord, string>;
  payments!: Table<PaymentLedgerRecord, string>;
  auditLogs!: Table<AuditLogRecord, string>;

  constructor() {
    super('FashionAuraAIStudioDB');
    this.version(1).stores({
      orders:
        'id, &orderNumber, customerId, primaryPhone, orderType, priority, paymentStatus,
fulfillmentStatus, orderDate, requestedDeliveryDate, earliestDispatchDate, district, courier,
isDraft, isDeleted, syncState, updatedAt',
      orderItems: 'id, orderId, productName, sku, size',
```

```
      customers: 'id, &primaryPhone, name, district, isRisky',
      payments: 'id, orderId, type, method, recordedAt',
      auditLogs: 'id, entityType, entityId, performedAt'
    });
  }
}

export const db = new FashionAuraDB();
```

STAGE 2: FORMATTERS, FINANCIAL ENGINE & BILINGUAL STRINGS

File: `src/utils/coreUtils.ts`

```
import type { PaymentStatus } from '../types/models';

const EN_TO_BN: Record<string, string> = {
  '0': '০', '1': '১', '2': '২', '3': '৩', '4': '৪',
  '5': '৫', '6': '৬', '7': '৭', '8': '৮', '9': '৯'
};
const BN_TO_EN: Record<string, string> = {
  '০': '0', '১': '1', '২': '2', '৩': '3', '৪': '4',
  '৫': '5', '৬': '6', '৭': '7', '৮': '8', '৯': '9'
};

export function toEnglishDigits(str: string): string {
  return (str || '').replace(/[০-৯]/g, (c) => BN_TO_EN[c] ?? c);
}

export function formatDigits(val: string | number, useBn: boolean): string {
  const s = String(val ?? '');
  if (!useBn) return toEnglishDigits(s);
  return s.replace(/[0-9]/g, (c) => EN_TO_BN[c] ?? c);
}

export function formatBDT(amount: number, useBn: boolean): string {
  const rounded = Math.round(amount || 0);
  const abs = Math.abs(rounded).toLocaleString('en-IN');
  return `${rounded < 0 ? '-' : ''}৳${formatDigits(abs, useBn)}`;
}

export function normalizeBDPhone(raw: string): string {
  const ascii = toEnglishDigits(raw).replace(/[\s\-().]/g, '');
  if (ascii.startsWith('+880')) return '0' + ascii.slice(4);
```

```
  if (ascii.startsWith('880') && ascii.length === 13) return '0' + ascii.slice(3);
  return ascii;
}

export function isValidBDPhone(raw: string): boolean {
  return /^01[3-9]\d{8}$/.test(normalizeBDPhone(raw));
}

export function parseIntSafe(raw: string): number {
  const clean = toEnglishDigits(raw).replace(/[^0-9]/g, '');
  return clean ? parseInt(clean, 10) : 0;
}

export function generateOrderId(): string {
  const now = new Date();
  const yy = String(now.getFullYear()).slice(-2);
  const mm = String(now.getMonth() + 1).padStart(2, '0');
  const dd = String(now.getDate()).padStart(2, '0');
  const rand = Math.floor(1000 + Math.random() * 9000);
  return `FA-${yy}${mm}${dd}-${rand}`;
}

export function calculateFinancials(params: {
  orderType: string;
  items: Array<{ unitPrice: number; quantity: number }>;
  deliveryCharge: number;
  additionalCharge: number;
  discount: number;
  netPaid: number;
}) {
  const subtotal = params.items.reduce(
    (sum, i) => sum + Math.max(0, Math.round(i.unitPrice)) * Math.max(1,
Math.round(i.quantity)),
    0
  );
  const rawTotal =
    params.orderType === 'Free'
      ? 0
      : subtotal +
        Math.round(params.deliveryCharge || 0) +
        Math.round(params.additionalCharge || 0) -
        Math.round(params.discount || 0);

  const totalBill = Math.max(0, rawTotal);
```

```
  const outstanding = totalBill - Math.round(params.netPaid || 0);
  const codAmount = Math.max(0, outstanding);

  let paymentStatus: PaymentStatus = 'Unpaid';
  if (params.orderType === 'Free' || (totalBill > 0 && params.netPaid >= totalBill)) {
    paymentStatus = 'Paid';
  } else if (params.netPaid > 0 && params.netPaid < totalBill) {
    paymentStatus = 'Partially Paid';
  }

  return { subtotal, totalBill, outstanding, codAmount, paymentStatus };
}

export const BD_DISTRICTS = [
  'Dhaka', 'Chattogram', 'Gazipur', 'Narayanganj', 'Sylhet', 'Rajshahi', 'Khulna',
  'Barishal', 'Rangpur', 'Mymensingh', 'Cumilla', 'Bogura', "Cox's Bazar", 'Jessore'
];

---

## STAGE 3: OFFLINE SMART PASTE PARSER & COURIER TEMPLATE ENGINE

### File: `src/services/smartPasteAndCourier.ts`
import { normalizeBDPhone, toEnglishDigits } from '../utils/coreUtils';
import type { OrderRecord, OrderItemRecord, ProductSize } from '../types/models';

export interface ParsedField<T> {
  value: T;
  status: 'Detected' | 'Ambiguous' | 'Not found';
}

export interface SmartPasteDraft {
  customerName: ParsedField<string>;
  primaryPhone: ParsedField<string>;
  fullAddress: ParsedField<string>;
  district: ParsedField<string>;
  productName: ParsedField<string>;
  size: ParsedField<ProductSize>;
  quantity: ParsedField<number>;
  unitPrice: ParsedField<number>;
  deliveryCharge: ParsedField<number>;
  advancePaid: ParsedField<number>;
  deliveryInstructions: ParsedField<string>;
  rawText: string;
```

```
}

/**
 * Deterministic, 100% Offline Order Message Parser (Bangla & English).
 * Never auto-creates an order; returns field statuses (Detected / Ambiguous / Not found).
 */
export function parseOrderMessageOffline(rawText: string): SmartPasteDraft {
  const normalizedDigits = toEnglishDigits(rawText);
  const lines = rawText
    .split(/\r?\n/)
    .map((l) => l.trim())
    .filter(Boolean);

  // 1. Extract BD Mobile Number
  const phoneRegex = /(?:\+?880|0)1[3-9]\d[\d\s\-]{7,10}/g;
  const phoneMatches = normalizedDigits.match(phoneRegex) || [];
  const cleanedPhone = phoneMatches[0] ? normalizeBDPhone(phoneMatches[0]) : '';

  // 2. Extract Size ONLY next to explicit size label (36-48)
  const sizeMatch =
normalizedDigits.match(/(?:size|সাইজ|বডি)\s*[:\-=]?\s*(36|38|40|42|44|46|48)/i);
  const detectedSize = (sizeMatch ? sizeMatch[1] : '40') as ProductSize;

  // 3. Extract Price & Delivery Charge
  const priceMatch =
normalizedDigits.match(/(?:price|দাম|মূল্য|বিল|tk|taka|৳)\s*[:\-=]?\s*(\d{3,5})/i);
  const deliveryMatch = normalizedDigits.match(/(?:delivery|ডেলিভারি)\s*[:\-=]?\s*(\d{2,3})/i);
  const advanceMatch =
normalizedDigits.match(/(?:advance|অ্যাডভান্স|অগ্রিম|বিকাশ|bkash)\s*[:\-=]?\s*(\d{2,5})/i);

  // 4. Extract Name, Address, Product by labels or line heuristics
  let nameVal = '';
  let addressVal = '';
  let productVal = '';
  let noteVal = '';

  for (const line of lines) {
    const lower = line.toLowerCase();
    if (/^(name|নাম)\s*[:\-]/i.test(line)) {
      nameVal = line.replace(/^(name|নাম)\s*[:\-]\s*/i, '').trim();
    } else if (/^(address|ঠিকানা)\s*[:\-]/i.test(line)) {
      addressVal = line.replace(/^(address|ঠিকানা)\s*[:\-]\s*/i, '').trim();
    } else if (/^(product|dress|ড্রেস|পণ্য)\s*[:\-]/i.test(line)) {
      productVal = line.replace(/^(product|dress|ড্রেস|পণ্য)\s*[:\-]\s*/i, '').trim();
```

```
    } else if (lower.includes('dispatch') || lower.includes('ডেলিভারি') || lower.includes('তারিখ')) {
      noteVal = line;
    }
  }

  if (!nameVal && lines.length > 0 && !/\d{5}/.test(lines[0])) {
    nameVal = lines[0];
  }
  if (!addressVal && lines.length > 1) {
    addressVal = lines.slice(1, 3).join(', ');
  }

  const isDhaka = /dhaka|ঢাকা|mirpur|dhanmondi|uttara|gulshan/i.test(rawText);

  return {
    customerName: { value: nameVal, status: nameVal ? 'Detected' : 'Not found' },
    primaryPhone: {
      value: cleanedPhone,
      status: cleanedPhone.length === 11 ? 'Detected' : phoneMatches.length > 0 ? 'Ambiguous' :
'Not found'
    },
    fullAddress: { value: addressVal, status: addressVal ? 'Detected' : 'Not found' },
    district: { value: isDhaka ? 'Dhaka' : 'Chattogram', status: isDhaka ? 'Detected' : 'Ambiguous' },
    productName: { value: productVal, status: productVal ? 'Detected' : 'Ambiguous' },
    size: { value: detectedSize, status: sizeMatch ? 'Detected' : 'Ambiguous' },
    quantity: { value: 1, status: 'Detected' },
    unitPrice: {
      value: priceMatch ? parseInt(priceMatch[1], 10) : 0,
      status: priceMatch ? 'Detected' : 'Not found'
    },
    deliveryCharge: {
      value: deliveryMatch ? parseInt(deliveryMatch[1], 10) : isDhaka ? 80 : 150,
      status: deliveryMatch ? 'Detected' : 'Ambiguous'
    },
    advancePaid: {
      value: advanceMatch ? parseInt(advanceMatch[1], 10) : 0,
      status: advanceMatch ? 'Detected' : 'Not found'
    },
    deliveryInstructions: { value: noteVal, status: noteVal ? 'Detected' : 'Not found' },
    rawText
  };
}

/**
```

```
 * Single Unified Courier & Share Template Engine.
 * Strictly excludes internal staff notes, risk flags, and audit history.
 */
export function generateCourierText(
  order: OrderRecord,
  items: OrderItemRecord[],
  lang: 'bn' | 'en' = 'bn'
): string {
  const itemsText = items
    .map(
      (i) =>
        `${i.productName} (${lang === 'bn' ? 'সাইজ' : 'Size'}: ${
          i.size === 'Other' ? i.customSizeText : i.size
        } × ${i.quantity})`
    )
    .join(', ');

  const cod = Math.max(0, order.outstanding);

  if (lang === 'bn') {
    const lines = [
      `অর্ডার আইডি: ${order.orderNumber}`,
      `নাম: ${order.customerName}`,
      `মোবাইল: ${order.primaryPhone}${order.alternatePhone ? `, ${order.alternatePhone}` : ''}`,
      `ঠিকানা: ${order.fullAddress}, ${order.thanaUpazila}, ${order.district}`,
      `পণ্য: ${itemsText}`,
      `মোট বিল: ৳${order.totalBill} (ডেলিভারি চার্জ: ৳${order.deliveryCharge})`,
      `ক্যাশ অন ডেলিভারি (COD): ${cod === 0 ? 'প্রিপেইড (৳০)' : `৳${cod}`}`
    ];
    if (order.requestedDeliveryDate) {
      lines.push(`ডেলিভারি তারিখ: ${order.requestedDeliveryDate}`);
    }
    if (order.customerDeliveryInstructions) {
      lines.push(`নির্দেশনা: ${order.customerDeliveryInstructions}`);
    }
    return lines.join('\n');
  }

  const enLines = [
    `Order ID: ${order.orderNumber}`,
    `Customer: ${order.customerName}`,
    `Phone: ${order.primaryPhone}${order.alternatePhone ? `, ${order.alternatePhone}` : ''}`,
    `Address: ${order.fullAddress}, ${order.thanaUpazila}, ${order.district}`,
    `Items: ${itemsText}`,
```

```
    `Total Bill: BDT ${order.totalBill} (Delivery: BDT ${order.deliveryCharge})`,
    `COD to Collect: ${cod === 0 ? 'Prepaid (BDT 0)' : `BDT ${cod}`}`
  ];
  if (order.requestedDeliveryDate) {
    enLines.push(`Requested Delivery: ${order.requestedDeliveryDate}`);
  }
  if (order.customerDeliveryInstructions) {
    enLines.push(`Note: ${order.customerDeliveryInstructions}`);
  }
  return enLines.join('\n');
}
```

STAGE 4: BACKUP, EXPORT (UTF-8 BOM CSV FOR EXCEL BANGLA) & RESTORE

File: `src/services/backupService.ts`

```
import { db } from '../db/database';
import { getDeliveryScheduleState } from '../types/models';

export async function exportOrdersToCSV(): Promise<void> {
  const orders = await db.orders.filter((o) => !o.isDeleted).toArray();
  const headers = [
    'Order ID',
    'Order Date',
    'Customer Name',
    'Phone',
    'Address',
    'District',
    'Thana',
    'Order Type',
    'Priority',
    'Payment Status',
    'Fulfillment Status',
    'Schedule State',
    'Earliest Dispatch',
    'Requested Delivery',
    'Total Bill',
    'Net Paid',
    'COD Outstanding',
    'Courier'
  ];

  const escapeCsv = (val: unknown) => `"${String(val ?? '').replace(/"/g, '""')}"`;
```

```
  const rows = orders.map((o) =>
    [
      o.orderNumber,
      o.orderDate.slice(0, 10),
      o.customerName,
      o.primaryPhone,
      o.fullAddress,
      o.district,
      o.thanaUpazila,
      o.orderType,
      o.priority,
      o.paymentStatus,
      o.fulfillmentStatus,
      getDeliveryScheduleState(o),
      o.earliestDispatchDate ?? '',
      o.requestedDeliveryDate ?? '',
      o.totalBill,
      o.netPaid,
      Math.max(0, o.outstanding),
      o.courier
    ]
      .map(escapeCsv)
      .join(',')
  );

  // UTF-8 BOM (\uFEFF) ensures Bangla characters open cleanly in Microsoft Excel
  const csvContent = '\uFEFF' + [headers.join(','), ...rows].join('\n');
  const blob = new Blob([csvContent], { type: 'text/csv;charset=utf-8;' });
  const url = URL.createObjectURL(blob);
  const link = document.createElement('a');
  link.href = url;
  link.download = `FashionAura_Orders_${new Date().toISOString().slice(0, 10)}.csv`;
  link.click();
  URL.revokeObjectURL(url);
}

export async function exportFullBackupJSON(): Promise<void> {
  const [orders, orderItems, customers, payments, auditLogs] = await Promise.all([
    db.orders.toArray(),
    db.orderItems.toArray(),
    db.customers.toArray(),
    db.payments.toArray(),
    db.auditLogs.toArray()
```

```
  ]);

  const payload = {
    app: 'Fashion Aura OMS',
    exportedAt: new Date().toISOString(),
    data: { orders, orderItems, customers, payments, auditLogs }
  };

  const blob = new Blob([JSON.stringify(payload, null, 2)], { type: 'application/json' });
  const url = URL.createObjectURL(blob);
  const link = document.createElement('a');
  link.href = url;
  link.download = `FashionAura_Backup_${new Date().toISOString().slice(0, 10)}.json`;
  link.click();
  URL.revokeObjectURL(url);
}

---

## STAGE 5: UNIFIED MOBILE-FIRST APPLICATION (`src/App.tsx`)

### File: `src/App.tsx`
import React, { useState, useMemo } from 'react';
import { useLiveQuery } from 'dexie-react-hooks';
import {
  ShoppingBag,
  PlusCircle,
  ClipboardPaste,
  Search,
  Copy,
  Trash2,
  RotateCcw,
  Download,
  AlertOctagon,
  Printer,
  CheckCircle2,
  Filter
} from 'lucide-react';
import { db } from './db/database';
import {
  getDeliveryScheduleState,
  type OrderRecord,
  type OrderType,
  type OrderPriority,
```

```
  type PaymentStatus,
  type FulfillmentStatus,
  type DeliveryScheduleState,
  type ProductSize,
  type CourierName
} from './types/models';
import {
  BD_DISTRICTS,
  calculateFinancials,
  formatBDT,
  formatDigits,
  generateOrderId,
  isValidBDPhone,
  normalizeBDPhone,
  parseIntSafe,
  toEnglishDigits
} from './utils/coreUtils';
import { generateCourierText, parseOrderMessageOffline } from
'./services/smartPasteAndCourier';
import { exportFullBackupJSON, exportOrdersToCSV } from './services/backupService';

export const App: React.FC = () => {
  const [lang, setLang] = useState<'bn' | 'en'>('bn');
  const [useBnDigits, setUseBnDigits] = useState(true);
  const [activeTab, setActiveTab] = useState<
    'dashboard' | 'orders' | 'create' | 'smartPaste' | 'trash' | 'backup'
  >('dashboard');

  // Universal Search & 5 Independent Dimension Filters
  const [searchQuery, setSearchQuery] = useState('');
  const [filterType, setFilterType] = useState<OrderType | 'All'>('All');
  const [filterPriority, setFilterPriority] = useState<OrderPriority | 'All'>('All');
  const [filterPayment, setFilterPayment] = useState<PaymentStatus | 'All'>('All');
  const [filterFulfillment, setFilterFulfillment] = useState<FulfillmentStatus | 'All'>('All');
  const [filterSchedule, setFilterSchedule] = useState<DeliveryScheduleState | 'All'>('All');

  // Order Form State
  const [editingOrderId, setEditingOrderId] = useState<string | null>(null);
  const [customerName, setCustomerName] = useState('');
  const [primaryPhone, setPrimaryPhone] = useState('');
  const [fullAddress, setFullAddress] = useState('');
  const [district, setDistrict] = useState('Dhaka');
  const [thanaUpazila, setThanaUpazila] = useState('Dhanmondi');
  const [orderType, setOrderType] = useState<OrderType>('Regular');
```

```
const [priority, setPriority] = useState<OrderPriority>('Normal');
const [fulfillmentStatus, setFulfillmentStatus] = useState<FulfillmentStatus>('New');
const [courier, setCourier] = useState<CourierName>('Steadfast');
const [requestedDeliveryDate, setRequestedDeliveryDate] = useState('');
const [earliestDispatchDate, setEarliestDispatchDate] = useState('');
const [deliveryCharge, setDeliveryCharge] = useState(80);
const [discount, setDiscount] = useState(0);
const [advancePaid, setAdvancePaid] = useState(0);
const [customerInstructions, setCustomerInstructions] = useState('');
const [internalNotes, setInternalNotes] = useState('');
const [productName, setProductName] = useState('');
const [sku, setSku] = useState('');
const [size, setSize] = useState<ProductSize>('40');
const [quantity, setQuantity] = useState(1);
const [unitPrice, setUnitPrice] = useState(1250);

// Smart Paste State
const [rawPasteText, setRawPasteText] = useState('');
const [toastMsg, setToastMsg] = useState<string | null>(null);

const allOrders = useLiveQuery(() => db.orders.reverse().sortBy('createdAt'), [], []);
const allItems = useLiveQuery(() => db.orderItems.toArray(), [], []);

const itemsByOrder = useMemo(() => {
  const map: Record<string, typeof allItems> = {};
  for (const item of allItems) {
    if (!map[item.orderId]) map[item.orderId] = [];
    map[item.orderId].push(item);
  }
  return map;
}, [allItems]);

const activeOrders = useMemo(() => allOrders.filter((o) => !o.isDeleted), [allOrders]);
const trashedOrders = useMemo(() => allOrders.filter((o) => o.isDeleted), [allOrders]);

// Real-Data Dashboard Metrics (Zero Fake Data)
const metrics = useMemo(() => {
  const todayIso = new Date().toISOString().slice(0, 10);
  return {
    total: activeOrders.length,
    newToday: activeOrders.filter((o) => o.orderDate.slice(0, 10) === todayIso).length,
    awaitingShip: activeOrders.filter((o) =>
      ['New', 'Confirmed', 'Preparing', 'Ready to Ship'].includes(o.fulfillmentStatus)
    ).length,
```

```
    urgent: activeOrders.filter((o) => o.priority === 'Urgent').length,
    scheduled: activeOrders.filter((o) => getDeliveryScheduleState(o) === 'Scheduled').length,
    preorders: activeOrders.filter((o) => o.orderType === 'Pre-order').length,
    unpaid: activeOrders.filter((o) => o.paymentStatus === 'Unpaid').length,
    codToCollect: activeOrders.reduce((sum, o) => sum + Math.max(0, o.outstanding), 0)
  };
}, [activeOrders]);

// Filtered & Searched Orders (Bilingual & Digit-Agnostic)
const filteredOrders = useMemo(() => {
  const q = toEnglishDigits(searchQuery).toLowerCase().trim();
  return activeOrders.filter((o) => {
    if (filterType !== 'All' && o.orderType !== filterType) return false;
    if (filterPriority !== 'All' && o.priority !== filterPriority) return false;
    if (filterPayment !== 'All' && o.paymentStatus !== filterPayment) return false;
    if (filterFulfillment !== 'All' && o.fulfillmentStatus !== filterFulfillment) return false;
    if (filterSchedule !== 'All' && getDeliveryScheduleState(o) !== filterSchedule) return false;

    if (!q) return true;
    const items = itemsByOrder[o.id] || [];
    const itemMatch = items.some(
      (i) =>
        i.productName.toLowerCase().includes(q) ||
        (i.sku && i.sku.toLowerCase().includes(q)) ||
        i.size.toLowerCase().includes(q)
    );
    return (
      o.orderNumber.toLowerCase().includes(q) ||
      o.customerName.toLowerCase().includes(q) ||
      o.primaryPhone.includes(q) ||
      o.fullAddress.toLowerCase().includes(q) ||
      o.district.toLowerCase().includes(q) ||
      itemMatch
    );
  });
}, [
  activeOrders,
  itemsByOrder,
  searchQuery,
  filterType,
  filterPriority,
  filterPayment,
  filterFulfillment,
  filterSchedule
```

```
  ]);

  const liveCalc = useMemo(
    () =>
      calculateFinancials({
        orderType,
        items: [{ unitPrice, quantity }],
        deliveryCharge,
        additionalCharge: 0,
        discount,
        netPaid: advancePaid
      }),
    [orderType, unitPrice, quantity, deliveryCharge, discount, advancePaid]
  );

  const showToast = (msg: string) => {
    setToastMsg(msg);
    setTimeout(() => setToastMsg(null), 3200);
  };

  const resetForm = () => {
    setEditingOrderId(null);
    setCustomerName('');
    setPrimaryPhone('');
    setFullAddress('');
    setProductName('');
    setSku('');
    setSize('40');
    setQuantity(1);
    setUnitPrice(1250);
    setAdvancePaid(0);
    setDiscount(0);
    setRequestedDeliveryDate('');
    setEarliestDispatchDate('');
    setCustomerInstructions('');
    setInternalNotes('');
  };

  const handleSaveOrder = async (e: React.FormEvent) => {
    e.preventDefault();
    if (!customerName.trim() || !isValidBDPhone(primaryPhone) || !fullAddress.trim() ||
!productName.trim()) {
      showToast(
        lang === 'bn'
```

```
      ? 'ক্রেতার নাম, ১১ সংখ্যার মোবাইল নম্বর, ঠিকানা ও পণ্যের নাম আবশ্যক!'
      : 'Name, valid 11-digit BD phone, address, and product name are required!'
  );
  return;
}

const now = new Date().toISOString();
const id = editingOrderId || crypto.randomUUID();
const orderNumber = editingOrderId
  ? (await db.orders.get(editingOrderId))?.orderNumber || generateOrderId()
  : generateOrderId();

const record: OrderRecord = {
  id,
  orderNumber,
  customerId: normalizeBDPhone(primaryPhone),
  customerName: customerName.trim(),
  primaryPhone: normalizeBDPhone(primaryPhone),
  fullAddress: fullAddress.trim(),
  district,
  thanaUpazila,
  orderType,
  priority,
  paymentStatus: liveCalc.paymentStatus,
  fulfillmentStatus,
  orderDate: now,
  requestedDeliveryDate: requestedDeliveryDate || undefined,
  earliestDispatchDate: earliestDispatchDate || undefined,
  source: 'WhatsApp',
  courier,
  subtotal: liveCalc.subtotal,
  deliveryCharge,
  additionalCharge: 0,
  discount,
  totalBill: liveCalc.totalBill,
  netPaid: advancePaid,
  outstanding: liveCalc.outstanding,
  customerDeliveryInstructions: customerInstructions.trim() || undefined,
  internalStaffNotes: internalNotes.trim() || undefined,
  isDraft: false,
  isDeleted: false,
  version: 1,
  createdAt: now,
  createdBy: 'Admin',
```

```
    updatedAt: now,
    updatedBy: 'Admin',
    syncState: 'pending'
  };

  await db.transaction('rw', [db.orders, db.orderItems, db.auditLogs], async () => {
    await db.orders.put(record);
    await db.orderItems.where('orderId').equals(id).delete();
    await db.orderItems.add({
      id: crypto.randomUUID(),
      orderId: id,
      productName: productName.trim(),
      sku: sku.trim() || undefined,
      size,
      quantity,
      unitPrice
    });
    await db.auditLogs.add({
      id: crypto.randomUUID(),
      entityType: 'order',
      entityId: id,
      action: editingOrderId ? 'UPDATED' : 'CREATED',
      details: `${orderNumber} - BDT ${liveCalc.totalBill}`,
      performedBy: 'Admin',
      performedAt: now
    });
  });

  resetForm();
  showToast(lang === 'bn' ? 'অর্ডার সফলভাবে সংরক্ষিত হয়েছে!' : 'Order saved to IndexedDB!');
  setActiveTab('orders');
};

const handleSmartFill = () => {
  if (!rawPasteText.trim()) return;
  const parsed = parseOrderMessageOffline(rawPasteText);
  setCustomerName(parsed.customerName.value);
  setPrimaryPhone(parsed.primaryPhone.value);
  setFullAddress(parsed.fullAddress.value);
  setDistrict(parsed.district.value);
  setProductName(parsed.productName.value || 'Three Piece');
  setSize(parsed.size.value);
  setUnitPrice(parsed.unitPrice.value || 1250);
  setDeliveryCharge(parsed.deliveryCharge.value);
```

```
  setAdvancePaid(parsed.advancePaid.value);
  setCustomerInstructions(parsed.deliveryInstructions.value);
  showToast(
    lang === 'bn'
      ? 'স্মার্ট ফিল সম্পন্ন! অনুগ্রহ করে যাচাই করে সেভ করুন।'
      : 'Smart Fill extracted! Review fields before saving.'
  );
  setActiveTab('create');
};

const handleCopyForCourier = async (order: OrderRecord) => {
  const items = itemsByOrder[order.id] || [];
  const text = generateCourierText(order, items, lang);
  await navigator.clipboard.writeText(text);
  await db.auditLogs.add({
    id: crypto.randomUUID(),
    entityType: 'share',
    entityId: order.id,
    action: 'COPIED_FOR_COURIER',
    details: order.orderNumber,
    performedBy: 'Admin',
    performedAt: new Date().toISOString()
  });
  showToast(lang === 'bn' ? 'কুরিয়ার টেক্সট কপি হয়েছে!' : 'Copied for Courier!');
};

const handleSoftDelete = async (orderId: string) => {
  await db.orders.update(orderId, { isDeleted: true, updatedAt: new Date().toISOString() });
  showToast(lang === 'bn' ? 'অর্ডার ট্র্যাশে সরানো হয়েছে।' : 'Moved to Trash.');
};

const handleRestore = async (orderId: string) => {
  await db.orders.update(orderId, { isDeleted: false, updatedAt: new Date().toISOString() });
  showToast(lang === 'bn' ? 'অর্ডার পুনরুদ্ধার হয়েছে!' : 'Order restored!');
};

return (
  <div className="min-h-screen bg-slate-50 text-slate-900 pb-24 font-sans">
    {/* HEADER */}
    <header className="sticky top-0 z-40 bg-pink-900 text-white px-4 py-3 shadow-md">
      <div className="max-w-3xl mx-auto flex items-center justify-between">
        <div>
          <h1 className="text-lg font-bold">
```

```
          {lang === 'bn' ? 'ফ্যাশন আউরা — অর্ডার ম্যানেজমেন্ট' : 'Fashion Aura — Order
Management'}
          </h1>
          <p className="text-xs text-pink-200">
           {lang === 'bn'
             ? 'লোকাল-অনলি মোড (IndexedDB সক্রিয়)'
             : 'Local-Only Mode (IndexedDB Active)'}
          </p>
         </div>
         <div className="flex gap-2">
          <button
           onClick={() => setUseBnDigits(!useBnDigits)}
           className="px-2.5 py-1 rounded bg-pink-800 text-xs font-bold"
          >
           {useBnDigits ? '১২৩' : '123'}
          </button>
          <button
           onClick={() => setLang(lang === 'bn' ? 'en' : 'bn')}
           className="px-3 py-1 rounded bg-white text-pink-900 text-xs font-bold"
          >
           {lang === 'bn' ? 'English' : 'বাংলা'}
          </button>
         </div>
        </div>
      </header>

      {toastMsg && (
       <div className="fixed top-16 left-1/2 -translate-x-1/2 z-50 bg-emerald-800 text-white px-4
py-2 rounded-xl shadow-lg flex items-center gap-2 text-sm font-bold">
         <CheckCircle2 className="w-4 h-4" />
         <span>{toastMsg}</span>
       </div>
      )}

      <main className="max-w-3xl mx-auto p-4 space-y-5">
       {/* REAL-DATA DASHBOARD */}
       {activeTab === 'dashboard' && (
        <div className="space-y-4">
         <div className="grid grid-cols-2 sm:grid-cols-4 gap-3">
          {[
           { label: lang === 'bn' ? 'মোট অর্ডার' : 'Total Orders', val: metrics.total },
           { label: lang === 'bn' ? 'আজকের নতুন' : 'New Today', val: metrics.newToday },
           { label: lang === 'bn' ? 'শিপমেন্টের অপেক্ষায়' : 'Awaiting Ship', val: metrics.awaitingShip },
           { label: lang === 'bn' ? 'জরুরি (Urgent)' : 'Urgent', val: metrics.urgent },
```

```
            { label: lang === 'bn' ? 'শিডিউলড' : 'Scheduled', val: metrics.scheduled },
            { label: lang === 'bn' ? 'প্রি-অর্ডার' : 'Pre-orders', val: metrics.preorders },
            { label: lang === 'bn' ? 'বাকি (Unpaid)' : 'Unpaid', val: metrics.unpaid },
            {
              label: lang === 'bn' ? 'মোট COD বাকি' : 'COD to Collect',
              val: formatBDT(metrics.codToCollect, useBnDigits)
            }
          ].map((card, i) => (
            <div
              key={i}
              onClick={() => setActiveTab('orders')}
              className="cursor-pointer bg-white p-3.5 rounded-2xl border border-slate-200
shadow-sm hover:border-pink-600"
            >
              <p className="text-xs text-slate-500 font-medium">{card.label}</p>
              <p className="text-xl font-extrabold text-pink-900 mt-1">
                {typeof card.val === 'number' ? formatDigits(card.val, useBnDigits) : card.val}
              </p>
            </div>
          ))}
        </div>

        <div className="flex gap-3">
          <button
            onClick={() => {
              resetForm();
              setActiveTab('create');
            }}
            className="flex-1 py-3 rounded-xl bg-pink-800 text-white font-bold flex items-center
justify-center gap-2 shadow"
          >
            <PlusCircle className="w-5 h-5" />
            {lang === 'bn' ? '+ নতুন অর্ডার' : '+ New Order'}
          </button>
          <button
            onClick={() => setActiveTab('smartPaste')}
            className="flex-1 py-3 rounded-xl bg-purple-800 text-white font-bold flex
items-center justify-center gap-2 shadow"
          >
            <ClipboardPaste className="w-5 h-5" />
            {lang === 'bn' ? 'স্মার্ট পেস্ট (Smart Paste)' : 'Smart Paste'}
          </button>
        </div>
      </div>
```

```
      )}

      {/* ORDERS LIST + UNIVERSAL SEARCH + 5 INDEPENDENT DIMENSION FILTERS */}
      {activeTab === 'orders' && (
        <div className="space-y-4">
          <div className="relative">
            <Search className="w-4 h-4 absolute left-3.5 top-3.5 text-slate-400" />
            <input
              type="text"
              value={searchQuery}
              onChange={(e) => setSearchQuery(e.target.value)}
              placeholder={
                lang === 'bn'
                  ? 'অর্ডার আইডি, নাম, মোবাইল নম্বর, ঠিকানা বা সাইজ দিয়ে খুঁজুন...'
                  : 'Search by Order ID, Name, Phone, Address, SKU or Size...'
              }
              className="w-full pl-10 pr-4 py-2.5 rounded-xl border border-slate-300 bg-white
text-sm"
            />
          </div>

          {/* 5 INDEPENDENT DIMENSION DROPDOWNS */}
          <div className="grid grid-cols-2 sm:grid-cols-5 gap-2 text-xs">
            <select
              value={filterType}
              onChange={(e) => setFilterType(e.target.value as OrderType | 'All')}
              className="p-2 rounded-lg border bg-white font-semibold"
            >
              <option value="All">Type: All</option>
              <option value="Regular">Regular</option>
              <option value="Pre-order">Pre-order</option>
              <option value="Free">Free</option>
              <option value="Exchange">Exchange</option>
            </select>
            <select
              value={filterPriority}
              onChange={(e) => setFilterPriority(e.target.value as OrderPriority | 'All')}
              className="p-2 rounded-lg border bg-white font-semibold"
            >
              <option value="All">Priority: All</option>
              <option value="Normal">Normal</option>
              <option value="Urgent">Urgent</option>
            </select>
            <select
```

```
            value={filterPayment}
            onChange={(e) => setFilterPayment(e.target.value as PaymentStatus | 'All')}
            className="p-2 rounded-lg border bg-white font-semibold"
          >
            <option value="All">Payment: All</option>
            <option value="Unpaid">Unpaid</option>
            <option value="Partially Paid">Partially Paid</option>
            <option value="Paid">Paid</option>
          </select>
          <select
            value={filterFulfillment}
            onChange={(e) => setFilterFulfillment(e.target.value as FulfillmentStatus | 'All')}
            className="p-2 rounded-lg border bg-white font-semibold"
          >
            <option value="All">Status: All</option>
            <option value="New">New</option>
            <option value="Confirmed">Confirmed</option>
            <option value="Ready to Ship">Ready to Ship</option>
            <option value="Shipped">Shipped</option>
            <option value="Delivered">Delivered</option>
          </select>
          <select
            value={filterSchedule}
            onChange={(e) => setFilterSchedule(e.target.value as DeliveryScheduleState | 'All')}
            className="p-2 rounded-lg border bg-white font-semibold"
          >
            <option value="All">Schedule: All</option>
            <option value="Scheduled">Scheduled</option>
            <option value="Unscheduled">Unscheduled</option>
          </select>
        </div>

        {filteredOrders.map((order) => {
          const scheduleState = getDeliveryScheduleState(order);
          const items = itemsByOrder[order.id] || [];
          return (
            <div
              key={order.id}
              className="bg-white p-4 rounded-2xl border border-slate-200 shadow-sm
space-y-2.5"
            >
              {order.earliestDispatchDate && (
                <div className="p-2 rounded-lg bg-red-600 text-white text-xs font-bold flex
items-center gap-1.5">
```

```
                <AlertOctagon className="w-4 h-4 shrink-0" />
                <span>Do Not Dispatch Before: {order.earliestDispatchDate}</span>
              </div>
            )}

            <div className="flex justify-between items-center">
              <span className="font-mono font-bold
text-pink-900">{order.orderNumber}</span>
              <span className="font-extrabold text-base">
                {formatBDT(order.totalBill, useBnDigits)} (COD:{' '}
                {formatBDT(Math.max(0, order.outstanding), useBnDigits)})
              </span>
            </div>

            <div className="text-sm">
              <p className="font-bold">
                {order.customerName} — <span
className="font-mono">{order.primaryPhone}</span>
              </p>
              <p className="text-xs text-slate-600">
                {order.fullAddress}, {order.thanaUpazila}, {order.district}
              </p>
              {items.map((it) => (
                <p key={it.id} className="text-xs font-semibold text-pink-800 mt-1">
                  • {it.productName} | Size: {it.size} × {it.quantity}
                </p>
              ))}
            </div>

            <div className="flex flex-wrap gap-1.5 text-[11px] font-bold">
              <span className="px-2 py-0.5 rounded bg-slate-100">{order.orderType}</span>
              <span
                className={`px-2 py-0.5 rounded ${
                  order.priority === 'Urgent' ? 'bg-red-100 text-red-800' : 'bg-slate-100'
                }`}
              >
                {order.priority}
              </span>
              <span className="px-2 py-0.5 rounded bg-emerald-100 text-emerald-900">
                {order.paymentStatus}
              </span>
              <span className="px-2 py-0.5 rounded bg-blue-100 text-blue-900">
                {order.fulfillmentStatus}
              </span>
```

```
                <span className="px-2 py-0.5 rounded bg-purple-100 text-purple-900">
                  {scheduleState}
                </span>
              </div>

              <div className="flex flex-wrap gap-2 pt-2 border-t text-xs font-bold">
                <button
                  onClick={() => handleCopyForCourier(order)}
                  className="px-3 py-1.5 rounded-lg bg-pink-900 text-white flex items-center
gap-1"
                >
                  <Copy className="w-3.5 h-3.5" />
                  Copy for Courier
                </button>
                <button
                  onClick={() => window.print()}
                  className="px-3 py-1.5 rounded-lg border border-slate-300 flex items-center
gap-1"
                >
                  <Printer className="w-3.5 h-3.5" />
                  Print Slip
                </button>
                <button
                  onClick={() => handleSoftDelete(order.id)}
                  className="px-3 py-1.5 rounded-lg text-red-700 border border-red-200 flex
items-center gap-1 ml-auto"
                >
                  <Trash2 className="w-3.5 h-3.5" />
                  Trash
                </button>
              </div>
            </div>
          );
        })}
      </div>
    )}

    {/* SMART PASTE / SMART FILL */}
    {activeTab === 'smartPaste' && (
      <div className="bg-white p-4 rounded-2xl border space-y-4">
        <h2 className="font-bold text-base text-pink-900">
          {lang === 'bn'
            ? 'স্মার্ট পেস্ট (WhatsApp / Facebook মেসেজ পেস্ট করুন)'
            : 'Smart Paste (Offline Order Message Parser)'}
```

```
          </h2>
          <textarea
            rows={6}
            value={rawPasteText}
            onChange={(e) => setRawPasteText(e.target.value)}
            placeholder={`নাম: তানিয়া আক্তার\nমোবাইল: ০১৭১২৩৪৫৬৭৮\nঠিকানা: বাড়ি ১২, রোড ৫, ধানমন্ডি, ঢাকা\nপণ্য: Organza Three Piece\nসাইজ: 42\nদাম: 1650\nঅগ্রিম: 200\n২৮ তারিখে ডেলিভারি, ২৬ তারিখের আগে পাঠাবেন না`}
            className="w-full p-3 rounded-xl border border-slate-300 text-sm"
          />
          <button
            onClick={handleSmartFill}
            className="w-full py-3 rounded-xl bg-purple-800 text-white font-bold"
          >
            {lang === 'bn' ? 'স্মার্ট ফিল করুন (রিভিউ ফর্মে যান)' : 'Smart Fill -> Review Form'}
          </button>
        </div>
      )}

      {/* CREATE / EDIT ORDER FORM */}
      {activeTab === 'create' && (
        <form onSubmit={handleSaveOrder} className="bg-white p-4 rounded-2xl border space-y-4">
          <h2 className="font-bold text-base text-pink-900">
            {lang === 'bn' ? 'অর্ডার ফর্ম (সব ফিল্ড যাচাই করুন)' : 'Create / Review Order'}
          </h2>
          <div className="grid grid-cols-1 sm:grid-cols-2 gap-3">
            <input
              type="text"
              placeholder="Customer Name *"
              value={customerName}
              onChange={(e) => setCustomerName(e.target.value)}
              className="px-3 py-2 rounded-lg border"
            />
            <input
              type="tel"
              placeholder="Phone 01XXXXXXXX *"
              value={primaryPhone}
              onChange={(e) => setPrimaryPhone(e.target.value)}
              className="px-3 py-2 rounded-lg border"
            />
          </div>
          <textarea
            rows={2}
```

```
  placeholder="Full Delivery Address *"
  value={fullAddress}
  onChange={(e) => setFullAddress(e.target.value)}
  className="w-full p-3 rounded-lg border"
/>
<div className="grid grid-cols-2 sm:grid-cols-4 gap-2">
  <select
    value={district}
    onChange={(e) => setDistrict(e.target.value)}
    className="p-2 rounded-lg border"
  >
    {BD_DISTRICTS.map((d) => (
      <option key={d} value={d}>
        {d}
      </option>
    ))}
  </select>
  <select
    value={orderType}
    onChange={(e) => setOrderType(e.target.value as OrderType)}
    className="p-2 rounded-lg border"
  >
    <option value="Regular">Regular</option>
    <option value="Pre-order">Pre-order</option>
    <option value="Free">Free</option>
    <option value="Exchange">Exchange</option>
  </select>
  <select
    value={priority}
    onChange={(e) => setPriority(e.target.value as OrderPriority)}
    className="p-2 rounded-lg border"
  >
    <option value="Normal">Normal</option>
    <option value="Urgent">Urgent</option>
  </select>
  <select
    value={courier}
    onChange={(e) => setCourier(e.target.value as CourierName)}
    className="p-2 rounded-lg border"
  >
    <option value="Steadfast">Steadfast</option>
    <option value="Pathao">Pathao</option>
    <option value="RedX">RedX</option>
  </select>
```

```
</div>

<div className="grid grid-cols-2 sm:grid-cols-4 gap-2">
  <input
    type="text"
    placeholder="Product Name *"
    value={productName}
    onChange={(e) => setProductName(e.target.value)}
    className="px-3 py-2 rounded-lg border col-span-2"
  />
  <select
    value={size}
    onChange={(e) => setSize(e.target.value as ProductSize)}
    className="p-2 rounded-lg border"
  >
    {(['36', '38', '40', '42', '44', '46', '48', 'Other'] as ProductSize[]).map((s) => (
      <option key={s} value={s}>
        Size {s}
      </option>
    ))}
  </select>
  <input
    type="text"
    placeholder="Unit Price ৳"
    value={unitPrice}
    onChange={(e) => setUnitPrice(parseIntSafe(e.target.value))}
    className="px-3 py-2 rounded-lg border"
  />
</div>

<div className="grid grid-cols-2 gap-2">
  <div>
    <label className="text-xs font-bold text-red-700">Do Not Dispatch Before</label>
    <input
      type="date"
      value={earliestDispatchDate}
      onChange={(e) => setEarliestDispatchDate(e.target.value)}
      className="w-full px-3 py-2 rounded-lg border border-red-300"
    />
  </div>
  <div>
    <label className="text-xs font-bold">Requested Delivery Date</label>
    <input
      type="date"
```

```
      value={requestedDeliveryDate}
      onChange={(e) => setRequestedDeliveryDate(e.target.value)}
      className="w-full px-3 py-2 rounded-lg border"
    />
  </div>
</div>

<div className="grid grid-cols-3 gap-2">
  <div>
    <label className="text-xs">Delivery ৳</label>
    <input
      type="text"
      value={deliveryCharge}
      onChange={(e) => setDeliveryCharge(parseIntSafe(e.target.value))}
      className="w-full px-3 py-2 rounded-lg border"
    />
  </div>
  <div>
    <label className="text-xs">Discount ৳</label>
    <input
      type="text"
      value={discount}
      onChange={(e) => setDiscount(parseIntSafe(e.target.value))}
      className="w-full px-3 py-2 rounded-lg border"
    />
  </div>
  <div>
    <label className="text-xs">Advance Paid ৳</label>
    <input
      type="text"
      value={advancePaid}
      onChange={(e) => setAdvancePaid(parseIntSafe(e.target.value))}
      className="w-full px-3 py-2 rounded-lg border"
    />
  </div>
</div>

<textarea
  rows={2}
  placeholder="Customer Delivery Instructions (Printed on Courier Slip)"
  value={customerInstructions}
  onChange={(e) => setCustomerInstructions(e.target.value)}
  className="w-full p-3 rounded-lg border"
/>
```

```
          <div className="p-3 rounded-xl bg-pink-950 text-white flex justify-between items-center
text-sm font-bold">
            <span>Total: {formatBDT(liveCalc.totalBill, useBnDigits)}</span>
            <span>COD Outstanding: {formatBDT(liveCalc.codAmount, useBnDigits)}</span>
          </div>

          <button
            type="submit"
            className="w-full py-3 rounded-xl bg-pink-800 text-white font-bold shadow"
          >
            {lang === 'bn' ? 'অর্ডার সেভ করুন' : 'Save Order'}
          </button>
        </form>
      )}

      {/* BACKUP & RESTORE / TRASH */}
      {activeTab === 'backup' && (
        <div className="bg-white p-4 rounded-2xl border space-y-4">
          <h2 className="font-bold text-base text-pink-900">Backup, Export & Trash</h2>
          <div className="flex flex-col sm:flex-row gap-3">
            <button
              onClick={exportOrdersToCSV}
              className="flex-1 py-3 px-4 rounded-xl bg-emerald-800 text-white font-bold flex
items-center justify-center gap-2"
            >
              <Download className="w-4 h-4" />
              Export Excel CSV (UTF-8 Bangla)
            </button>
            <button
              onClick={exportFullBackupJSON}
              className="flex-1 py-3 px-4 rounded-xl bg-slate-800 text-white font-bold flex
items-center justify-center gap-2"
            >
              <Download className="w-4 h-4" />
              Full JSON Backup
            </button>
          </div>

          <div className="pt-4 border-t space-y-2">
            <h3 className="font-bold text-sm">Trash ({trashedOrders.length})</h3>
            {trashedOrders.map((o) => (
              <div key={o.id} className="flex justify-between items-center p-2 border rounded-lg
text-xs">
```

```
              <span>
                {o.orderNumber} — {o.customerName}
              </span>
              <button
                onClick={() => handleRestore(o.id)}
                className="px-2.5 py-1 rounded bg-emerald-100 text-emerald-900 font-bold flex
items-center gap-1"
              >
                <RotateCcw className="w-3.5 h-3.5" />
                Restore
              </button>
            </div>
          ))}
        </div>
      </div>
    )}
  </main>

  {/* BOTTOM NAVIGATION */}
  <nav className="fixed bottom-0 left-0 right-0 bg-white border-t flex justify-around py-1.5
z-40 text-xs font-bold">
    <button
      onClick={() => setActiveTab('dashboard')}
      className={activeTab === 'dashboard' ? 'text-pink-800' : 'text-slate-500'}
    >
      Dashboard
    </button>
    <button
      onClick={() => setActiveTab('orders')}
      className={activeTab === 'orders' ? 'text-pink-800' : 'text-slate-500'}
    >
      Orders ({activeOrders.length})
    </button>
    <button
      onClick={() => {
        resetForm();
        setActiveTab('create');
      }}
      className={activeTab === 'create' ? 'text-pink-800' : 'text-slate-500'}
    >
      + Add
    </button>
    <button
      onClick={() => setActiveTab('smartPaste')}
```

```
          className={activeTab === 'smartPaste' ? 'text-pink-800' : 'text-slate-500'}
        >
          Smart Paste
        </button>
        <button
          onClick={() => setActiveTab('backup')}
          className={activeTab === 'backup' ? 'text-pink-800' : 'text-slate-500'}
        >
          Backup/Trash
        </button>
      </nav>
    </div>
  );
};
```